>_ MCMP113 · Week 4

Arithmetic, Logical and Relational Operators

Computing with values: expressions, precedence, the Math class and boolean logic

Asst. Prof. Hasan Oğuz, PhD
Introduction to Basic Algorithms and Programming · Istanbul Okan University
Vocational School · 2026–2027 Fall

How to use these notes

These notes follow the Week 4 slides but explain every idea in more detail. All programs are in the folder `code/` (one console project per example, e.g. `04_01_ArithmeticCalculator`); the listings below are included directly from those files, and every console window shows a real run (typed input is shown in amber). **Predict before you run**: for every expression, write down what you expect, then run the code and compare. The exercises at the end (with ⋆–⋆⋆⋆ difficulty) are the best preparation for the predict-the-output questions of the midterm.

Contents

1 Expressions, operands and operators 2

2 Arithmetic operators 2

3 Precedence, assignment and increment 9

4 The Math class 12

5 Relational and logical operators 16

6 Common mistakes 19

7 Summary 20

8 Glossary (English – Turkish) 20

9 Exercises 20

10 References 22

Learning outcomes

After this week you should be able to:

1. use the arithmetic operators `+ - * / %` and predict whether a result is `int` or `double`;
2. explain integer division, the sign of `%` for negative numbers, and division by zero for `int` and `double`;
3. evaluate any expression step by step using operator precedence and associativity;
4. use compound assignment (`+= -= *= /= %=`) and increment/decrement (prefix vs postfix) and trace them;
5. apply `Math.Pow`, `Sqrt`, `Abs`, `Round`, `Floor`, `Ceiling`, `Max`, `Min` correctly, including banker's rounding;
6. write relational and logical expressions of type `bool`, use short-circuit evaluation as a guard, and apply De Morgan's laws.

>_ 1 Expressions, operands and operators

In Week 3 we learned to *store* values in typed variables. This week we *compute* with them. An **expression** is any piece of code that produces a value, for example `price * qty + shipping`. The values an operator works on are its **operands**; the symbol is the **operator**.

Definition – operator arity

A **unary** operator has one operand (`-x`, `!ok`, `++i`); a **binary** operator has two (`a + b`, `a < b`); the only **ternary** operator in C# is the conditional operator `c ? a : b`.

Every expression has a type. The key rule of the week is:

> The *types of the operands* decide the *type of the result*.
> `7 / 2` is `int` (value 3); `7 / 2.0` is `double` (value 3.5).

Assignment `=` is also an operator, but it has the lowest priority: in `total = price * qty + shipping;` the whole right-hand side is evaluated first, and only then is the result stored in `total`.

>_ 2 Arithmetic operators

2.1 The five operators

Op.	Meaning	int example	Prints	double example	Prints
`+`	addition	`7 + 2`	9	`7.0 + 2`	9
`-`	subtraction	`7 - 2`	5	`7.0 - 2`	5
`*`	multiplication	`7 * 2`	14	`7.0 * 2`	14
`/`	division	`7 / 2`	3	`7.0 / 2`	3.5
`%`	remainder	`7 % 2`	1	`7.5 % 2`	1.5

If both operands are `int`, the operation is performed in `int` and the result is `int`. If at least one operand is `double`, the other is converted (*widened*) to `double` and the result is `double`. Note that C# prints the `double` value 9.0 simply as `9`. There is no power operator in C# (`^` means something else); use `Math.Pow` (Section 4).

2.2 Integer division vs real division

For integers, `/` computes the quotient and *drops the fractional part*; it truncates *toward zero*, so `-7 / 2` is -3 (not -4). To get a real result, at least one operand must be `double` *before* the division happens.

Listing 1: Division rules (`code/04_02_DivisionRules`)

```
// MCMP113 · Week 4 · Example 2 — Division rules
// Integer vs real division, the remainder of negative numbers, division by zero.
Console.WriteLine("--- integer vs real division ---");
Console.WriteLine($"7 / 2           = {7 / 2}");
Console.WriteLine($"7 / 2.0         = {7 / 2.0}");
Console.WriteLine($"(double)7 / 2   = {(double)7 / 2}");
Console.WriteLine($"(double)(7 / 2) = {(double)(7 / 2)}");
Console.WriteLine($"-7 / 2          = {-7 / 2}   (truncated toward zero)");

Console.WriteLine("--- remainder: the sign follows the dividend ---");
Console.WriteLine($" 7 %  3 = {7 % 3}");
Console.WriteLine($"-7 %  3 = {-7 % 3}");
Console.WriteLine($" 7 % -3 = {7 % -3}");
int x = -7;
Console.WriteLine($"((x % 3) + 3) % 3 = {((x % 3) + 3) % 3}   (always 0, 1 or 2)");

Console.WriteLine("--- division by zero ---");
double zero = 0.0;
Console.WriteLine($" 1.0 / 0.0 = {1.0 / zero}");
Console.WriteLine($"-1.0 / 0.0 = {-1.0 / zero}");
Console.WriteLine($" 0.0 / 0.0 = {0.0 / zero}");
// int n = 0; Console.WriteLine(1 / n);   // int: program stops with
    DivideByZeroException
```

Output

```
--- integer vs real division ---
7 / 2           = 3
7 / 2.0         = 3.5
(double)7 / 2   = 3.5
(double)(7 / 2) = 3
-7 / 2          = -3   (truncated toward zero)
--- remainder: the sign follows the dividend ---
 7 %  3 = 1
-7 %  3 = -1
 7 % -3 = 1
((x % 3) + 3) % 3 = 2   (always 0, 1 or 2)
--- division by zero ---
 1.0 / 0.0 = Infinity
-1.0 / 0.0 = -Infinity
 0.0 / 0.0 = NaN
```

Common mistake: casting too late

`(double)(7 / 2)` is `3`, not `3.5`: the parentheses force the integer division first, and the cast only converts the finished result 3 into 3.0. Cast an *operand*: `(double)7 / 2` or `(double)a / b`.

2.3 The remainder operator `%`

For integers a and $b \neq 0$ the quotient and the remainder always satisfy the **division identity**

$$a = (a/b) \cdot b + (a \,\%\, b), \qquad \text{e.g.} \quad 17 = 3 \cdot 5 + 2.$$

Typical uses of `%`:

- **parity**: `n % 2 == 0` is `true` exactly for even `n`;
- **digits**: `n % 10` is the last digit, `n / 10` drops the last digit;
- **wrap-around**: `(hour + 5) % 24` is the hour five hours later, e.g. $(22 + 5) \bmod 24 = 3$;
- **divisibility**: `year % 4 == 0` (leap years, Section 5.6).

2.4 `%` with negative numbers

Because `/` truncates toward zero, the identity forces the remainder to have the **sign of the dividend** (the left operand):

a	b	a / b	a % b	check: (a/b)*b + a%b
7	3	2	1	2*3 + 1 = 7
-7	3	-2	-1	(-2)*3 + (-1) = -7
7	-3	-2	1	(-2)*(-3) + 1 = 7
-7	-3	2	-1	2*(-3) + (-1) = -7

In mathematics, $-7 \bmod 3 = 2$ (the result is always in $\{0, 1, \ldots, m-1\}$), but in C# `-7 % 3` is -1. (Python's `%` behaves like the mathematical one, so the two languages disagree here.) When you need a result in $0, \ldots, m-1$ for any integer x and $m > 0$, use the **safe modulo**

$$r = \big((x \,\%\, m) + m\big) \,\%\, m, \qquad \text{e.g.} \quad \big((-7 \,\%\, 3) + 3\big) \,\%\, 3 = (-1 + 3) \,\%\, 3 = 2.$$

2.5 Division by zero

- **`int`**: dividing by a zero *variable* stops the program at run time with a `DivideByZeroException` (message: "Attempted to divide by zero."). The same happens for `%`. If the zero is a literal (`10 / 0`), the compiler refuses the program already: `error CS0020: Division by constant zero`.
- **`double`**: follows the IEEE 754 standard and does not crash: $1.0/0.0$ = `Infinity`, $-1.0/0.0$ = `-Infinity`, $0.0/0.0$ = `NaN` ("Not a Number"). These values then spread silently through later calculations.

Running Example 1 below with $b = 0$ prints the first three lines and then stops:

Output (b = 0)

```
a + b = 17
a - b = 17
a * b = 0
```

followed by `Unhandled exception. System.DivideByZeroException: Attempted to divide by zero.` In Week 5 we will prevent this with an `if` that checks the divisor first.

2.6 Worked example 1: arithmetic calculator

Problem. Read two integers a and b and print $a+b$, $a-b$, $a\cdot b$, the integer quotient, the remainder and the real quotient; verify the division identity.

Listing 2: Arithmetic calculator (`code/04_01_ArithmeticCalculator`)

```
// MCMP113 · Week 4 · Example 1 — Arithmetic calculator
// The five arithmetic operators on int, and integer division vs real division.
Console.Write("First integer  a = ");
int a = int.Parse(Console.ReadLine());
Console.Write("Second integer b = ");
int b = int.Parse(Console.ReadLine());

Console.WriteLine($"a + b = {a + b}");
Console.WriteLine($"a - b = {a - b}");
Console.WriteLine($"a * b = {a * b}");
Console.WriteLine($"a / b = {a / b}   (quotient)");
Console.WriteLine($"a % b = {a % b}   (remainder)");
Console.WriteLine($"(double)a / b = {(double)a / b:F4}");

// quotient * divisor + remainder always gives back a
Console.WriteLine($"check: {a / b} * {b} + {a % b} = {a / b * b + a % b}");
```

Run 1

```
First integer  a = 17
Second integer b = 5
a + b = 22
a - b = 12
a * b = 85
a / b = 3   (quotient)
a % b = 2   (remainder)
(double)a / b = 3.4000
check: 3 * 5 + 2 = 17
```

Run 2

```
First integer  a = -17
Second integer b = 5
a + b = -12
a - b = -22
a * b = -85
a / b = -3   (quotient)
a % b = -2   (remainder)
(double)a / b = -3.4000
check: -3 * 5 + -2 = -17
```

2.7 Worked example 2: digits of a three-digit number

Problem. Read a three-digit number and print its hundreds, tens and ones digit, the digit sum and the reversed number. (In pseudocode, **div** is integer division and mod the remainder.)

Algorithm 1: digits of a three-digit number

1 **START**
2 **READ** n
3 $h \leftarrow n$ **div** 100
4 $t \leftarrow (n$ **div** $10) \bmod 10$
5 $o \leftarrow n \bmod 10$
6 **PRINT** $h, t, o, h+t+o, 100\cdot o + 10\cdot t + h$
7 **END**

Listing 3: Digit extraction (`code/04_03_DigitExtraction`)

```
// MCMP113 · Week 4 · Example 3 — Digits of a three-digit number
// Uses / 10 (drop the last digit) and % 10 (keep the last digit).
Console.Write("Enter a three-digit number: ");
int n = int.Parse(Console.ReadLine());

int hundreds = n / 100;       // 472 / 100 = 4
int tens = n / 10 % 10;       // 472 / 10 = 47, 47 % 10 = 7
int ones = n % 10;            // 472 % 10 = 2

int digitSum = hundreds + tens + ones;
int reversed = ones * 100 + tens * 10 + hundreds;

Console.WriteLine($"hundreds = {hundreds}, tens = {tens}, ones = {ones}");
Console.WriteLine($"digit sum = {digitSum}");
Console.WriteLine($"reversed  = {reversed}");
```

Output

```
Enter a three-digit number: 472
hundreds = 4, tens = 7, ones = 2
digit sum = 13
reversed  = 274
```

Note `n / 10 % 10`: `/` and `%` have the same precedence and are evaluated left to right, so it means `(n / 10) % 10`. For 472: $472/10 = 47$, $47 \bmod 10 = 7$.

2.8 Worked example 3: seconds to days, hours, minutes and seconds

Problem. Convert a number of seconds into days, hours, minutes and seconds. The pattern "divide to get the big unit, keep the remainder for the rest" appears everywhere (money change, unit conversion).

Algorithm 2: seconds to d h:m:s

1 **START**
2 **READ** $total$
3 $d \leftarrow total$ **div** 86400
4 $h \leftarrow (total \bmod 86400)$ **div** 3600
5 $m \leftarrow (total \bmod 3600)$ **div** 60
6 $s \leftarrow total \bmod 60$
7 **PRINT** d, h, m, s
8 **END**

Listing 4: Seconds to d h:m:s (`code/04_04_SecondsToHms`)

```
// MCMP113 · Week 4 · Example 4 — Seconds to days, hours, minutes, seconds
// Repeated use of / and % to split one quantity into units.
Console.Write("Total seconds: ");
int total = int.Parse(Console.ReadLine());

int days = total / 86400;             // 86400 s in a day
int hours = total % 86400 / 3600;     // what is left after whole days
int minutes = total % 3600 / 60;
int seconds = total % 60;

Console.WriteLine($"{total} s = {days} d {hours} h {minutes} min {seconds} s");
```

```
12 Console.WriteLine($"clock format: {days}d {hours:D2}:{minutes:D2}:{seconds:D2}");
```

Expression (total = 100000)	Value
`total / 86400`	1
`total % 86400`	13600
`total % 86400 / 3600`	3
`total % 3600`	2800
`total % 3600 / 60`	46
`total % 60`	40

Output

```
Total seconds: 100000
100000 s = 1 d 3 h 46 min 40 s
clock format: 1d 03:46:40
```

Check: $1 \cdot 86400 + 3 \cdot 3600 + 46 \cdot 60 + 40 = 86400 + 10800 + 2760 + 40 = 100000$. The format `{hours:D2}` pads an integer to two digits ($3 \to$ `03`).

2.9 Worked example 4: clock arithmetic

Problem. Given the current time (hour, minute) and a number of minutes Δ (possibly negative), print the new time on a 24-hour clock.

Idea. Convert the time to "minutes since midnight" ($0 \le now \le 1439$), add Δ, wrap into $0, \ldots, 1439$ with the safe modulo, convert back with `/ 60` and `% 60`.

Listing 5: Clock arithmetic (`code/04_05_ClockArithmetic`)

```
1  // MCMP113 · Week 4 · Example 5 — Clock arithmetic
2  // Time after N minutes (N may be negative): wrap-around with the safe modulo.
3  Console.Write("Current hour (0-23): ");
4  int hour = int.Parse(Console.ReadLine());
5  Console.Write("Current minute (0-59): ");
6  int minute = int.Parse(Console.ReadLine());
7  Console.Write("Minutes to add (negative = go back): ");
8  int delta = int.Parse(Console.ReadLine());
9
10 const int MinutesPerDay = 24 * 60;             // 1440
11 int now = hour * 60 + minute;                  // minutes since midnight
12 int raw = now + delta;
13 int wrapped = ((raw % MinutesPerDay) + MinutesPerDay) % MinutesPerDay;
14
15 Console.WriteLine($"raw minutes = {raw}, raw % 1440 = {raw % MinutesPerDay}");
16 Console.WriteLine($"New time: {wrapped / 60:D2}:{wrapped % 60:D2}");
```

Run 1: 22:45 plus 150 minutes

```
Current hour (0-23): 22
Current minute (0-59): 45
Minutes to add (negative = go back): 150
raw minutes = 1515, raw % 1440 = 75
New time: 01:15
```

Run 2: 00:20 minus 45 minutes

```
Current hour (0-23): 0
Current minute (0-59): 20
Minutes to add (negative = go back): -45
raw minutes = -25, raw % 1440 = -25
New time: 23:35
```

In run 2 the plain remainder -25 is useless as a time; the safe modulo gives $(-25 + 1440) \bmod 1440 = 1415 = 23 \cdot 60 + 35$, i.e. 23:35 on the previous day.

2.10 Worked example 5: T.C. Kimlik No check digits

In Week 1 we met the rules for the two check digits of the 11-digit Turkish identity number $d_1 d_2 \ldots d_{11}$:

$$d_{10} = \Big(7\,(d_1 + d_3 + d_5 + d_7 + d_9) - (d_2 + d_4 + d_6 + d_8)\Big) \bmod 10,$$
$$d_{11} = \Big(d_1 + d_2 + \cdots + d_{10}\Big) \bmod 10.$$

We also noted that the bracket in d_{10} can be negative, so C#'s `%` needs the safe-modulo fix. The program reads the first nine digits as one `int` (at most $999\,999\,999 < 2\,147\,483\,647$) and extracts each digit with `/` and `%`.

Algorithm 3: check digits

1 **START**
2 **READ** $first9$ ▷ nine digits $d_1 \ldots d_9$
3 $d_k \leftarrow (first9 \textbf{ div } 10^{9-k}) \bmod 10$ for $k = 1, \ldots, 9$
4 $odd \leftarrow d_1 + d_3 + d_5 + d_7 + d_9$; $\quad even \leftarrow d_2 + d_4 + d_6 + d_8$
5 $d_{10} \leftarrow ((7 \cdot odd - even) \bmod 10 + 10) \bmod 10$
6 $d_{11} \leftarrow (odd + even + d_{10}) \bmod 10$
7 **PRINT** $first9, d_{10}, d_{11}$
8 **END**

Listing 6: Check digits (`code/04_10_KimlikCheckDigits`)

```
// MCMP113 · Week 4 · Example 10 — T.C. Kimlik No check digits d10 and d11
// Digit extraction with / and %, and the safe modulo for a negative number.
Console.Write("First nine digits: ");
int first9 = int.Parse(Console.ReadLine());

int d1 = first9 / 100_000_000;
int d2 = first9 / 10_000_000 % 10;
int d3 = first9 / 1_000_000 % 10;
int d4 = first9 / 100_000 % 10;
int d5 = first9 / 10_000 % 10;
int d6 = first9 / 1_000 % 10;
int d7 = first9 / 100 % 10;
int d8 = first9 / 10 % 10;
int d9 = first9 % 10;

int oddSum = d1 + d3 + d5 + d7 + d9;
int evenSum = d2 + d4 + d6 + d8;
int raw = 7 * oddSum - evenSum;
int d10 = ((raw % 10) + 10) % 10;             // safe modulo: result is 0..9
int d11 = (oddSum + evenSum + d10) % 10;

Console.WriteLine($"odd sum = {oddSum}, even sum = {evenSum}, 7*odd - even = {raw}");
Console.WriteLine($"raw % 10 = {raw % 10}  ->  d10 = {d10}, d11 = {d11}");
Console.WriteLine($"Full number: {first9}{d10}{d11}   (first digit non-zero: {d1 != 0})");
```

Run 1 (made-up digits from Week 1)

```
First nine digits: 123456789
odd sum = 25, even sum = 20, 7*odd - even = 155
raw % 10 = 5  ->  d10 = 5, d11 = 0
Full number: 12345678950   (first digit non-zero: True)
```

Run 2 (made-up digits, negative bracket)

```
First nine digits: 190909090
odd sum = 1, even sum = 36, 7*odd - even = -29
raw % 10 = -9  ->  d10 = 1, d11 = 8
Full number: 19090909018   (first digit non-zero: True)
```

In run 2, `-29 % 10` is -9, which is not a digit; the safe modulo gives $(-9+10) \bmod 10 = 1$. Both numbers are invented: never use a real person's identity number in exercises.

>_ 3 Precedence, assignment and increment

3.1 Precedence and associativity

When an expression contains several operators, **precedence** decides which one is applied first, and **associativity** decides the order among operators of the *same* precedence. The table lists the operators of this course from highest to lowest precedence (the complete C# table also contains shift, bitwise and null operators; see the references).

Level	Operators	Category	Associativity
1 (first)	`x++ x-- f(x) a.b`	primary / postfix	left to right
2	`+x -x !x ++x --x (int)x`	unary / cast	right to left
3	`* / %`	multiplicative	left to right
4	`+ -`	additive	left to right
5	`< > <= >=`	relational	left to right
6	`== !=`	equality	left to right
7	`&&`	conditional AND	left to right
8	`||`	conditional OR	left to right
9	`?:`	conditional	right to left
10 (last)	`= += -= *= /= %=`	assignment	right to left

Parentheses override the table and are always allowed; use them whenever an expression could be misread.

Step by step. Evaluate $3+4*2-6/3\%2$: first all of `* / %` from left to right, then `+ -` from left to right.

Step	Expression	Next operation
0	`3 + 4 * 2 - 6 / 3 % 2`	`4 * 2` (leftmost of `* / %`)
1	`3 + 8 - 6 / 3 % 2`	`6 / 3`
2	`3 + 8 - 2 % 2`	`2 % 2`
3	`3 + 8 - 0`	`3 + 8` (leftmost of `+ -`)
4	`11 - 0`	`11 - 0`
5	`11`	done (C# prints 11)

3.2 The Week 1 average bug

Listing 7: Operator pitfalls (`code/04_14_OperatorPitfalls`)

```
// MCMP113 · Week 4 · Example 14 — Operator pitfalls
// Precedence (Week 1 average bug), integer division, string + number, overflow.
Console.Write("Grade 1: ");
int g1 = int.Parse(Console.ReadLine());
Console.Write("Grade 2: ");
int g2 = int.Parse(Console.ReadLine());
Console.Write("Grade 3: ");
int g3 = int.Parse(Console.ReadLine());

Console.WriteLine($"g1 + g2 + g3 / 3     = {g1 + g2 + g3 / 3}      (wrong: / before +)");
Console.WriteLine($"(g1 + g2 + g3) / 3   = {(g1 + g2 + g3) / 3}       (integer division)")
    ;
Console.WriteLine($"(g1 + g2 + g3) / 3.0 = {(g1 + g2 + g3) / 3.0:F2}    (correct)");

Console.WriteLine("Sum: " + g1 + g2);       // text first -> joins as text
Console.WriteLine("Sum: " + (g1 + g2));     // parentheses -> adds first
Console.WriteLine(g1 + g2 + " is the sum"); // left to right: adds, then joins

int big = int.MaxValue;
Console.WriteLine($"int.MaxValue + 1 = {big + 1}   (overflow wraps around)");
Console.WriteLine($"(long)big + 1    = {(long)big + 1}");
// Console.WriteLine(checked(big + 1));   // would stop with OverflowException
```

Output

```
Grade 1: 60
Grade 2: 70
Grade 3: 85
g1 + g2 + g3 / 3     = 158      (wrong: / before +)
(g1 + g2 + g3) / 3   = 71       (integer division)
(g1 + g2 + g3) / 3.0 = 71.67    (correct)
Sum: 6070
Sum: 130
130 is the sum
int.MaxValue + 1 = -2147483648   (overflow wraps around)
(long)big + 1    = 2147483648
```

- `g1 + g2 + g3 / 3` divides only `g3`: $60 + 70 + \lfloor 85/3 \rfloor = 60 + 70 + 28 = 158$.
- `(g1 + g2 + g3) / 3` has correct parentheses but still uses integer division: $\lfloor 215/3 \rfloor = 71$.
- `(g1 + g2 + g3) / 3.0` is real division: $215/3 = 71.\overline{6}$, printed with `F2` as `71.67`.

All three versions compile: the compiler checks syntax and types, not your intention. Only testing with values you can check by hand reveals such bugs.

3.3 String + number

`+` is *overloaded*: with two numbers it adds, but as soon as one operand is a `string` it joins text. Since `+` is evaluated left to right, `"Sum: " + g1 + g2` is `("Sum: " + 60) + 70`, i.e. `"Sum: 6070"`, whereas `g1 + g2 + " is the sum"` first adds the two numbers. Parentheses or string interpolation (`$"Sum: {g1 + g2}"`) avoid the problem. A `char` is a number in arithmetic: `'A' + 1` is the `int` 66 (the code of `'A'` is 65).

3.4 Integer overflow (recap from Week 3)

An `int` holds values from $-2\,147\,483\,648$ to $2\,147\,483\,647$. Going past the end wraps around silently: `int.MaxValue + 1` gives `int.MinValue`. Remedies: use `long` (`(long)big + 1` is correct), or write `checked(big + 1)`, which turns the silent wrap into a `System.OverflowException`. If the overflowing expression consists only of constants, e.g. `int.MaxValue + 1`, the compiler reports `error CS0220: The operation overflows at compile time in checked mode`.

3.5 Compound assignment

`x op= y` means `x = x op (y)`. The whole right-hand side is evaluated first: `c *= 3 + 1` is `c = c * (3 + 1)`, not `c = c * 3 + 1`.

Operator	Example	Means	score
`=`	`score = 10`	`(store 10)`	10
`+=`	`score += 5`	`score = score + 5`	15
`-=`	`score -= 3`	`score = score - 3`	12
`*=`	`score *= 2`	`score = score * 2`	24
`/=`	`score /= 5`	`score = score / 5`	4
`%=`	`score %= 3`	`score = score % 3`	1

Each row starts from the previous value of `score`; `/=` on an `int` is integer division ($24/5 = 4$).

3.6 Increment and decrement: prefix vs postfix

`i++` and `++i` both add 1 to `i` (and `i--`, `--i` subtract 1). They differ only in the *value of the expression*:

Prefix vs postfix

Postfix `i++`: the expression has the *old* value of `i`; then `i` is incremented.
Prefix `++i`: `i` is incremented first; the expression has the *new* value.

Listing 8: Increment, decrement, compound assignment (`code/04_13_IncrementDecrement`)

```
// MCMP113 · Week 4 · Example 13 — Increment, decrement and compound assignment
// Prefix (++i) changes first and then gives the value; postfix (i++) gives the old value.

int i = 5;
int a = i++;                        // a gets 5, then i becomes 6
Console.WriteLine($"a = i++  ->  a = {a}, i = {i}");

i = 5;
int b = ++i;                        // i becomes 6, then b gets 6
Console.WriteLine($"b = ++i  ->  b = {b}, i = {i}");

int j = 3;
int c = j-- * 10;                   // uses 3, then j becomes 2
Console.WriteLine($"c = j-- * 10  ->  c = {c}, j = {j}");

int score = 10;
score += 5;  Console.WriteLine($"score += 5  ->  {score}");
score -= 3;  Console.WriteLine($"score -= 3  ->  {score}");
score *= 2;  Console.WriteLine($"score *= 2  ->  {score}");
score /= 5;  Console.WriteLine($"score /= 5  ->  {score}   (integer division)");
score %= 3;  Console.WriteLine($"score %= 3  ->  {score}");
```

Output

```
a = i++   ->  a = 5, i = 6
b = ++i   ->  b = 6, i = 6
c = j-- * 10   ->  c = 30, j = 2
score += 5   ->  15
score -= 3   ->  12
score *= 2   ->  24
score /= 5   ->  4   (integer division)
score %= 3   ->  1
```

a = i++;	i	a
start	5	?
a ← old i	5	5
i ← i + 1	6	5

b = ++i;	i	b
start	5	?
i ← i + 1	6	?
b ← new i	6	6

When ++ stands alone as a statement (`i++;`), prefix and postfix are identical, and that is how you should normally use them. Inside larger expressions they make code hard to read; we practise them only so that you can read other people's code and answer exam questions.

>_ 4 The Math class

The class `Math` provides the usual mathematical functions. Each one is called with the class name in front, for example `Math.Sqrt(16)`.

Method	Meaning	Example	Prints
`Math.Pow(x, y)`	x^y	`Math.Pow(2, 10)`	`1024`
`Math.Sqrt(x)`	$\sqrt{x}$	`Math.Sqrt(16)`	`4`
`Math.Abs(x)`	$\lvert x \rvert$	`Math.Abs(-7.5)`	`7.5`
`Math.Max(a, b)`	the larger	`Math.Max(3, 7)`	`7`
`Math.Min(a, b)`	the smaller	`Math.Min(3, 7)`	`3`
`Math.Round(x)`	nearest integer (ties to even)	`Math.Round(2.5)`	`2`
`Math.Floor(x)`	round down, $\lfloor x \rfloor$	`Math.Floor(-2.5)`	`-3`
`Math.Ceiling(x)`	round up, $\lceil x \rceil$	`Math.Ceiling(2.1)`	`3`
`Math.PI`	π	`Math.PI`	`3.141592653589793`

Math.Pow and Math.Sqrt return double

`int k = Math.Pow(2, 3);` does not compile: `error CS0266: Cannot implicitly convert type 'double' to 'int'. An explicit conversion exists (are you missing a cast?)`. Write `int k = (int)Math.Pow(2, 3);`. `Math.Abs`, `Max` and `Min` keep the type of their arguments (`int` in, `int` out). For a square, `x * x` is simpler than `Math.Pow(x, 2)`.

4.1 Rounding

`Math.Round(x)` rounds to the nearest integer, and a tie (a fractional part of exactly .5) goes to the **even** neighbour: $0.5 \to 0$, $1.5 \to 2$, $2.5 \to 2$, $3.5 \to 4$. This **banker's rounding** avoids a systematic upward bias when many rounded values are added. For the "school" rule (ties away from zero) write `Math.Round(x, MidpointRounding.AwayFromZero)`. `Floor` always goes down, `Ceiling` always up, and a cast `(int)x` cuts off the fraction (toward zero); the three differ for negative numbers.

Listing 9: The Math class (`code/04_11_MathClass`)

```
// MCMP113 · Week 4 · Example 11 — The Math class and rounding
// Round (banker's vs away from zero), Floor, Ceiling, cast, Abs, Pow, Sqrt, Max, Min.
Console.Write("Enter a real number x: ");
double x = double.Parse(Console.ReadLine());

Console.WriteLine($"Math.Round(x)                  = {Math.Round(x)}");
Console.WriteLine($"Math.Round(x, AwayFromZero)    = {Math.Round(x, MidpointRounding.
    AwayFromZero)}");
Console.WriteLine($"Math.Floor(x)                  = {Math.Floor(x)}");
Console.WriteLine($"Math.Ceiling(x)                = {Math.Ceiling(x)}");
Console.WriteLine($"(int)x                         = {(int)x}");
Console.WriteLine($"Math.Abs(x)                    = {Math.Abs(x)}");
Console.WriteLine($"Math.Pow(x, 2)                 = {Math.Pow(x, 2)}");
Console.WriteLine($"Math.Sqrt(Math.Abs(x))         = {Math.Sqrt(Math.Abs(x)):F4}");
Console.WriteLine($"Math.Max(x, 0), Math.Min(x, 0) = {Math.Max(x, 0)}, {Math.Min(x, 0)}");
```

Output (x = -2.7)

```
Enter a real number x: -2.7
Math.Round(x)                  = -3
Math.Round(x, AwayFromZero)    = -3
Math.Floor(x)                  = -3
Math.Ceiling(x)                = -2
(int)x                         = -2
Math.Abs(x)                    = 2.7
Math.Pow(x, 2)                 = 7.290000000000001
Math.Sqrt(Math.Abs(x))         = 1.6432
Math.Max(x, 0), Math.Min(x, 0) = 0, -2.7
```

Running the same program with other inputs gives:

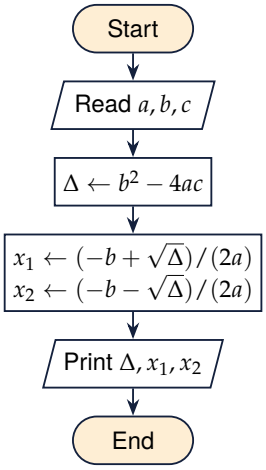
x	Round(x)	Round(x, AwayFromZero)	Floor(x)	Ceiling(x)	(int)x
2.5	2	3	2	3	2
3.5	4	4	3	4	3
-2.5	-2	-3	-3	-2	-2
-2.7	-3	-3	-3	-2	-2

Notice `Math.Pow(-2.7, 2)` prints `7.290000000000001`: the tiny error comes from the binary representation of 2.7 (Section 5.2). To round to k decimals use `Math.Round(x, k)`, e.g. `Math.Round(3.14159, 2)` is `3.14`. Be careful with values such as 1.005, which are stored slightly below the decimal value: `Math.Round(1.005, 2)` prints `1`. For money, use `decimal` (Week 3).

4.2 Worked example 6: quadratic equation

Problem. Read a, b, c and compute the discriminant $\Delta = b^2 - 4ac$ and the roots $x_{1,2} = \dfrac{-b \pm \sqrt{\Delta}}{2a}$.

This week the flowchart is a pure sequence (we cannot branch yet); `Math.Sqrt` of a negative number returns `NaN`, and the conditional operator `?:` chooses the message. In Week 5 an `if–else if` ladder will handle $\Delta > 0$, $\Delta = 0$ and $\Delta < 0$ separately.



Listing 10: Quadratic roots (`code/04_07_QuadraticRoots`)

```
// MCMP113 · Week 4 · Example 7 — Quadratic equation ax² + bx + c = 0
// Discriminant, Math.Sqrt and correct parentheses; NaN appears when there are no real roots.
Console.Write("a = ");
double a = double.Parse(Console.ReadLine());
Console.Write("b = ");
double b = double.Parse(Console.ReadLine());
Console.Write("c = ");
double c = double.Parse(Console.ReadLine());

double delta = b * b - 4 * a * c;
double x1 = (-b + Math.Sqrt(delta)) / (2 * a);    // parentheses are essential
double x2 = (-b - Math.Sqrt(delta)) / (2 * a);

Console.WriteLine($"delta = {delta}");
Console.WriteLine(delta >= 0 ? "Real roots exist." : "No real roots (Sqrt of a negative is NaN).");
Console.WriteLine($"x1 = {x1:F4}");
Console.WriteLine($"x2 = {x2:F4}");
```

$x^2 - 3x + 2 = 0$

```
a = 1
b = -3
c = 2
delta = 1
Real roots exist.
x1 = 2.0000
x2 = 1.0000
```

$x^2 + 2x + 5 = 0$

```
a = 1
b = 2
c = 5
delta = -16
No real roots (Sqrt of a negative is NaN).
x1 = NaN
x2 = NaN
```

Check run 1: $x_1 = 2$: $4 - 6 + 2 = 0$ and $x_2 = 1$: $1 - 3 + 2 = 0$. The parentheses in `(-b + Math.Sqrt(delta)) / (2 * a)` are essential: `-b + Math.Sqrt(delta) / 2 * a` would compute $-b + \frac{\sqrt{\Delta}}{2} \cdot a$.

4.3 Worked example 7: compound interest

With principal P, annual rate r (as a fraction), n compounding periods per year and t years, the final amount is

$$A = P\left(1 + \frac{r}{n}\right)^{nt}.$$

Listing 11: Compound interest (`code/04_08_CompoundInterest`)

```
// MCMP113 · Week 4 · Example 8 — Compound interest A = P(1 + r/n)^(nt)
// Math.Pow, operator precedence and converting a percentage to a rate.
Console.Write("Principal P (TL): ");
double principal = double.Parse(Console.ReadLine());
Console.Write("Annual interest rate (%): ");
double ratePercent = double.Parse(Console.ReadLine());
Console.Write("Compounding periods per year n: ");
int n = int.Parse(Console.ReadLine());
Console.Write("Years t: ");
int t = int.Parse(Console.ReadLine());

double r = ratePercent / 100;                          // 40 % -> 0.40
double amount = principal * Math.Pow(1 + r / n, n * t);
double simple = principal * (1 + r * t);               // simple interest, for
    comparison

Console.WriteLine($"Compound amount : {amount:F2} TL");
Console.WriteLine($"Interest earned : {amount - principal:F2} TL");
Console.WriteLine($"Simple interest : {simple:F2} TL");
```

Output

```
Principal P (TL): 10000
Annual interest rate (%): 40
Compounding periods per year n: 12
Years t: 2
Compound amount : 21966.99 TL
Interest earned : 11966.99 TL
Simple interest : 18000.00 TL
```

Here $A = 10000\,(1 + 0.4/12)^{24} \approx 21\,966.99$ TL, versus $10000\,(1 + 0.4 \cdot 2) = 18\,000$ TL with simple interest. A quick sanity check with easy numbers: $P = 1000$, 10 %, $n = 1$, $t = 3$ gives `1331.00` $= 1000 \cdot 1.1^3$ (verified with the same program).

Integer trap

If the rate were read into an `int`, `ratePercent / 100` would be `40 / 100 = 0`, and the program would silently print $A = P$. Reading it as `double` (or dividing by `100.0`) avoids this.

4.4 Worked example 8: distance between two points

For $P_1 = (x_1, y_1)$ and $P_2 = (x_2, y_2)$ the Euclidean distance is $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$, the "taxi" (Manhattan) distance is $|x_2 - x_1| + |y_2 - y_1|$ and the midpoint is $\left(\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2}\right)$.

Listing 12: Distance (`code/04_09_Distance`)

```
// MCMP113 · Week 4 · Example 9 — Distance between two points
// Math.Sqrt, Math.Pow and Math.Abs in a geometric formula.
Console.Write("x1 = ");
double x1 = double.Parse(Console.ReadLine());
Console.Write("y1 = ");
double y1 = double.Parse(Console.ReadLine());
Console.Write("x2 = ");
double x2 = double.Parse(Console.ReadLine());
Console.Write("y2 = ");
double y2 = double.Parse(Console.ReadLine());
```

```
11
12 double dx = x2 - x1;
13 double dy = y2 - y1;
14 double distance = Math.Sqrt(dx * dx + dy * dy);
15 double samePow = Math.Sqrt(Math.Pow(dx, 2) + Math.Pow(dy, 2));   // same value
16 double manhattan = Math.Abs(dx) + Math.Abs(dy);                  // "taxi" distance
17
18 Console.WriteLine($"Euclidean distance = {distance:F3} (with Pow: {samePow:F3})");
19 Console.WriteLine($"Manhattan distance = {manhattan:F3}");
20 Console.WriteLine($"Midpoint = ({(x1 + x2) / 2:F2}, {(y1 + y2) / 2:F2})");
```

Output

```
x1 = 1
y1 = 2
x2 = 4
y2 = 6
Euclidean distance = 5.000 (with Pow: 5.000)
Manhattan distance = 7.000
Midpoint = (2.50, 4.00)
```

Check: $\Delta x = 3$, $\Delta y = 4$, $\sqrt{9+16} = 5$ (the 3–4–5 right triangle).

>_ 5 Relational and logical operators

5.1 Relational operators

A comparison is an expression of type `bool`; C# prints its value as `True` or `False`.

C#	Pseudocode	Meaning	Example	Prints
`==`	$=$	equal to	`5 == 5`	`True`
`!=`	$\neq$	not equal to	`5 != 3`	`True`
`<`	$<$	less than	`3 < 2`	`False`
`>`	$>$	greater than	`7 > 2`	`True`
`<=`	$\leq$	less than or equal	`5 <= 5`	`True`
`>=`	$\geq$	greater than or equal	`4 >= 9`	`False`

= assigns, == compares

In pseudocode $\leftarrow$ assigns and $=$ compares; in C# `=` assigns and `==` compares. `bool same = (x = 5);` does not compile: `error CS0029: Cannot implicitly convert type 'int' to 'bool'`.

5.2 Comparing doubles

A `double` stores numbers in binary, and most decimal fractions (such as 0.1) have no exact binary representation, just as $1/3 = 0.333\ldots$ has no exact decimal one. The tiny errors can become visible:

```
1 double sum = 0.1 + 0.2;
2 Console.WriteLine(sum);                          // 0.30000000000000004
3 Console.WriteLine(sum == 0.3);                   // False
4 Console.WriteLine(Math.Abs(sum - 0.3) < 1e-9);   // True
5 Console.WriteLine(0.1m + 0.2m == 0.3m);          // True (decimal)
```

(The outputs in the comments were produced by running these lines.) **Rule:** never compare `double` values with `==`; test whether their distance is below a small tolerance ε such as 10^{-9}. The `decimal` type stores base-10 digits exactly, which is why money uses `decimal`.

5.3 Logical operators and truth tables

p	q	p && q	p || q	!p
False	False	False	False	True
False	True	False	True	True
True	False	False	True	False
True	True	True	True	False

`&&` (AND, *ve*) is true only when both operands are true; `||` (OR, *veya*) is true when at least one is true; `!` (NOT, *değil*) inverts. Precedence: `!` before `&&` before `||`, so `a || b && c` means `a || (b && c)`. Write the parentheses yourself whenever you mix `&&` and `||`.

Listing 13: Truth table printer (`code/04_12_TruthTable`)

```
// MCMP113 · Week 4 · Example 12 — Truth table printer
// &&, ||, ! for all four input combinations, De Morgan's law, short-circuit guard.
bool p, q;
Console.WriteLine("p      q      p&&q   p||q   !p     !(p&&q)==(!p||!q)");

p = false; q = false;
Console.WriteLine($"{p,-6} {q,-6} {p && q,-6} {p || q,-6} {!p,-6} {!(p && q) == (!p || !q)
    }");
p = false; q = true;
Console.WriteLine($"{p,-6} {q,-6} {p && q,-6} {p || q,-6} {!p,-6} {!(p && q) == (!p || !q)
    }");
p = true; q = false;
Console.WriteLine($"{p,-6} {q,-6} {p && q,-6} {p || q,-6} {!p,-6} {!(p && q) == (!p || !q)
    }");
p = true; q = true;
Console.WriteLine($"{p,-6} {q,-6} {p && q,-6} {p || q,-6} {!p,-6} {!(p && q) == (!p || !q)
    }");

// short-circuit: when b is 0, a / b is never evaluated, so no crash
int a = 10, b = 0;
bool bigRatio = b != 0 && a / b > 2;
Console.WriteLine($"b != 0 && a / b > 2  ->  {bigRatio}");
```

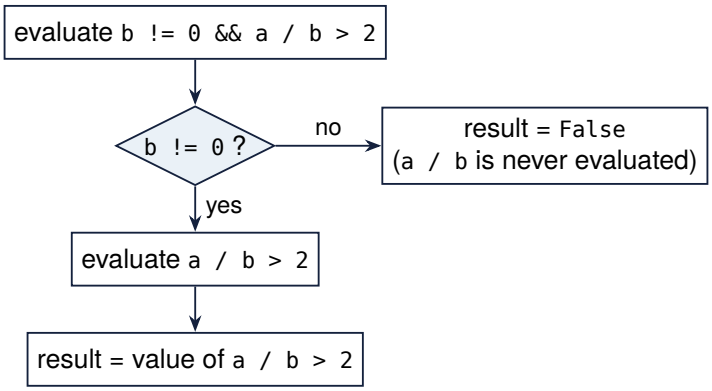
Output

```
p      q      p&&q   p||q   !p     !(p&&q)==(!p||!q)
False  False  False  False  True   True
False  True   False  True   True   True
True   False  False  True   False  True
True   True   True   True   False  True
b != 0 && a / b > 2  ->  False
```

The four rows are written out by hand; in Week 6 a loop will do this repetition for us. The format item `{p,-6}` prints the value left-aligned in a field of width 6.

5.4 Short-circuit evaluation

`&&` and `||` evaluate the left operand first and skip the right operand when the result is already known: `false && …` is `false` and `true || …` is `true`. This makes a built-in **guard**:



With `b = 0` the program above prints `False` and does not crash. The order matters: `a / b > 2 && b != 0` divides first and crashes when `b` is 0. (The single `&` and `|` also work on `bool` but never short-circuit; we do not use them in this course.)

5.5 De Morgan's laws and range tests

De Morgan's laws

`!(A && B)` ≡ `!A || !B` `!(A || B)` ≡ `!A && !B`

To negate a combined condition, negate each part and swap `&&` ↔ `||`.

Range tests. Mathematics writes $0 \le x \le 100$, but C# parses `0 <= x <= 100` as `(0 <= x) <= 100`, i.e. `bool <= int`, and reports `error CS0019: Operator '<=' cannot be applied to operands of type 'bool' and 'int'`. Write `x >= 0 && x <= 100`. "Not between 18 and 65 (inclusive)" is `!(age >= 18 && age <= 65)`, which by De Morgan equals `age < 18 || age > 65`.

5.6 Worked example 9: leap year

Gregorian rule: a year is a leap year if it is divisible by 4 but not by 100, or if it is divisible by 400.

Algorithm 4: leap year as one boolean expression

1 **START**
2 **READ** y
3 $isLeap \leftarrow (y \bmod 4 = 0$ **AND** $y \bmod 100 \neq 0)$ **OR** $y \bmod 400 = 0$
4 **PRINT** $isLeap$
5 **END**

Listing 14: Leap year (`code/04_06_LeapYear`)

```
// MCMP113 · Week 4 · Example 6 — Leap year as one boolean expression
// Relational (==, !=) and logical (&&, ||) operators produce a bool.
Console.Write("Year: ");
int year = int.Parse(Console.ReadLine());

bool divisibleBy4 = year % 4 == 0;
bool divisibleBy100 = year % 100 == 0;
bool divisibleBy400 = year % 400 == 0;
```

```
9  bool isLeap = (divisibleBy4 && !divisibleBy100) || divisibleBy400;
10
11 Console.WriteLine($"divisible by 4: {divisibleBy4}, by 100: {divisibleBy100}, by 400: {
       divisibleBy400}");
12 Console.WriteLine($"isLeap = {isLeap}");
13 Console.WriteLine($"{year} is {(isLeap ? "a leap year (366 days)" : "not a leap year (365
       days)")}");
```

Output

```
Year: 1900
divisible by 4: True, by 100: True, by 400: False
isLeap = False
1900 is not a leap year (365 days)
```

Test table (every row produced by running the program):

year	% 4 == 0	% 100 == 0	% 400 == 0	isLeap	Case
2024	True	False	False	True	ordinary leap year
1900	True	True	False	False	century, not leap
2000	True	True	True	True	divisible by 400
2026	False	False	False	False	ordinary year

5.7 The conditional operator `?:`

`condition ? valueIfTrue : valueIfFalse` evaluates the condition and then *only one* of the two values. We use it this week only to choose between two texts, e.g. `isLeap ? "a leap year (366 days)" : "not a leap year (365 days)"`. Inside an interpolation hole it must be put in parentheses: `{(isLeap ? "…" : "…")}`. The general tool for decisions, `if–else`, is next week's topic.

>_ 6 Common mistakes

Six operator bugs

1. **Integer division**: expecting `7 / 2` to be 3.5. Fix: `7 / 2.0` or `(double)a / b`.
2. **Missing parentheses**: `g1 + g2 + g3 / 3`. Fix: `(g1 + g2 + g3) / 3.0`.
3. **Chained comparison**: `0 <= x <= 100` (error CS0019). Fix: `x >= 0 && x <= 100`.
4. **`=` instead of `==`**: `bool ok = (x = 5);` (error CS0029). Fix: `x == 5`.
5. **`==` on doubles**: `0.1 + 0.2 == 0.3` is `False`. Fix: `Math.Abs(a - b) < 1e-9`.
6. **Negative remainder**: `-7 % 3` is -1. Fix: `((x % m) + m) % m`.

Further traps: `"Sum: " + a + b` joins text; `(double)(a / b)` casts too late; `Math.Round(2.5)` is 2 (banker's rounding); `int` overflow wraps around silently; `int k = Math.Pow(2, 3);` needs a cast.

>_ 7 Summary

- `int op int` gives `int`; one `double` operand makes the result `double`. Integer `/` truncates toward zero.
- `a == (a / b) * b + a % b`; the sign of `%` follows the dividend; the safe modulo `((x % m) + m) % m` is always in $0..m-1$.
- Division by zero: `int` throws `DivideByZeroException`, `double` gives `Infinity` or `NaN`.
- Precedence: unary → `* / %` → `+ -` → relational → `== !=` → `&&` → `||` → `?:` → assignment. Same level: left to right (except unary, `?:` and assignment).
- `x op= y` means `x = x op (y)`; `i++` yields the old value, `++i` the new one.
- `Math.Pow/Sqrt` return `double`; `Math.Round` uses banker's rounding by default; for ties away from zero pass `MidpointRounding.AwayFromZero` as the second argument.
- Relational operators give `bool`; never use == on doubles; `&&`/`||` short-circuit; De Morgan: `!(A && B)` ≡ `!A || !B`.

>_ 8 Glossary (English – Turkish)

English	Türkçe	English	Türkçe
operator	işleç (operatör)	operand	işlenen
expression	ifade	integer division	tam sayı bölmesi
quotient	bölüm	remainder	kalan
dividend / divisor	bölünen / bölen	division by zero	sıfıra bölme
precedence	öncelik	associativity	birleşme yönü
parentheses	parantez	assignment	atama
compound assignment	bileşik atama	increment / decrement	artırma / azaltma
prefix / postfix	önek / sonek	overflow	taşma
square root	karekök	power, exponent	kuvvet, üs
rounding	yuvarlama	absolute value	mutlak değer
relational operator	ilişkisel işleç	logical operator	mantıksal işleç
truth table	doğruluk tablosu	short-circuit evaluation	kısa devre değerlendirme
condition	koşul	conditional operator	koşul işleci
tolerance	tolerans	discriminant	diskriminant

>_ 9 Exercises

9.1 In-class exercises

E1 ⋆ Coins

Read an amount in kuruş (e.g. 385). Print how many 1 TL (100 kr), 50, 25, 10, 5 and 1 kuruş coins make up the amount with the fewest coins. Use only `/` and `%` (and compound assignment).

E2 ⋆ Four digits

Read a four-digit number. Print its four digits, their sum and the reversed number (e.g. 2026 → 6202).

E3 ⋆⋆ Heron's formula

Read the side lengths a, b, c of a triangle (as `double`). Print the half-perimeter $s = (a+b+c)/2$ and the area $\sqrt{s(s-a)(s-b)(s-c)}$ with two decimals. Test with $3, 4, 5$ (area 6).

E4 ⋆⋆ Boolean checks

Read an integer x. Print four `bool` values: `isEven`, `isPositive`, `isTwoDigit` ($10 \le |x| \le 99$) and `isMultipleOf3Or5`.

9.2 Homework

H1 ⋆⋆ Kimlik validator

Read an 11-digit number as `long`. Compute d_{10} and d_{11} from its first nine digits and print a single `bool`: is the number valid ($d_1 \neq 0$ and both check digits match)? Use made-up numbers only.

H2 ⋆⋆ Time difference

Read two clock times (hour and minute of each, one number per line). Print how many minutes pass from the first to the second, wrapping past midnight (22:30 → 01:15 is 165 minutes).

H3 ⋆⋆ Loan payment

A loan P is repaid in n equal monthly payments with monthly rate i = annual rate/12. The payment is

$$M = \frac{P\, i\, (1+i)^n}{(1+i)^n - 1}.$$

Read P, the annual rate in percent and n; print M and the total paid, with two decimals.

H4 ⋆–⋆⋆⋆ Predict the output

Write down exactly what each line prints, then check by running the code. Lines 9 and 10 belong together (`int k = 4;` is declared before line 9).

```
1  Console.WriteLine(23 / 4);
2  Console.WriteLine(23 % 4);
3  Console.WriteLine(23 / 4.0);
4  Console.WriteLine(-23 % 4);
5  Console.WriteLine(2 + 10 / 4 * 2);
6  Console.WriteLine(7 - 2 * 3 + 8 / 3);
7  Console.WriteLine("x" + 3 * 2);
8  Console.WriteLine(3 + 4 + "x" + 3 + 4);
9  Console.WriteLine(k++ * 2);
10 Console.WriteLine(k);
11 Console.WriteLine(!(3 < 5) && true);
12 Console.WriteLine(Math.Round(4.5));
```

Bonus ⋆⋆⋆: what does `Console.WriteLine(Math.Ceiling((double)(7 / 2)));` print, and why does `Math.Ceiling(7 / 2)` not even compile?

9.3 Extra practice

X1 ⋆⋆ Swap without a temporary variable

Swap the values of two `int` variables `a` and `b` using only `+` and `-` (no third variable). Why is the Week 3 version with a temporary variable still preferable?

X2 ⋆⋆ Round to the nearest multiple of 5

For a non-negative integer v, compute the nearest multiple of 5 using only integer arithmetic. Test with 42, 43 and 47.

X3 ⋆⋆ Day of the week

Day 1 of a year is a Monday (weekday number 1, Sunday = 7). Write an expression for the weekday number of day N. Which weekday is day 100?

X4 ⋆ BMI

Compute the body-mass index BMI $= w/h^2$ for $w = 70$ kg and $h = 1.75$ m and print it with one decimal in two ways: `{bmi:F1}` and `Math.Round(bmi, 1)`.

X5 ⋆⋆⋆ Ceiling division

A web page shows `perPage` items per page. Write two expressions for the number of pages needed for `items` items: one with integers only and one with `Math.Ceiling`. Test with 23 items, 5 per page.

X6 ⋆⋆ De Morgan

Rewrite without the outer `!`: (a) `!(x > 0 && y > 0)`, (b) `!(grade < 0 || grade > 100)`, (c) `!(a == b || !c)`.

>_ 10 References

- Arithmetic operators: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/arithmetic-operators
- Operator precedence and associativity (full table): https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/
- Boolean logical operators: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/boolean-logical-operators
- Comparison operators: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/comparison-operators
- Assignment and compound assignment: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/assignment-operator
- Conditional operator `?:`: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/conditional-operator
- `checked` and `unchecked`: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/checked-and-unchecked
- `Math` class: https://learn.microsoft.com/en-us/dotnet/api/system.math
- `Math.Round` and `MidpointRounding`: https://learn.microsoft.com/en-us/dotnet/api/system.math.round
- Floating-point numeric types: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/floating-point-numeric-types